

You can order print and ebook versions of *Think Bayes 2e* from Bookshop.org and [Amazon](http://Amazon.com).

Estimating Proportions

```
# install empiricaldist if necessary
```

```
try:
    import empiricaldist
except ImportError:
    !pip install empiricaldist
    import empiricaldist
```

```
# Get utils.py
```

```
from os.path import basename, exists
```

```
def download(url):
    filename = basename(url)
    if not exists(filename):
        from urllib.request import urlretrieve
        local, _ = urlretrieve(url, filename)
        print('Downloaded ' + local)
```

```
download('https://github.com/AllenDowney/ThinkBayes2/raw/master/soln/utils.py')
```

```
from utils import set_pyplot_params
set_pyplot_params()
```

In the previous chapter we solved the 101 Bowls Problem, and I admitted that it is not really about guessing which bowl the cookies came from; it is about estimating proportions.

In this chapter, we take another step toward Bayesian statistics by solving the Euro problem. We'll start with the same prior distribution, and we'll see that the update is the same, mathematically. But I will argue that it is a different problem, philosophically, and use it to introduce two defining elements of Bayesian statistics: choosing prior distributions, and using probability to represent the unknown.

The Euro Problem

In *Information Theory, Inference, and Learning Algorithms*, David MacKay poses this problem:

"A statistical statement appeared in *The Guardian* on Friday January 4, 2002:

When spun on edge 250 times, a Belgian one-euro coin came up heads 140 times and tails 110. 'It looks very suspicious to me,' said Barry Blight, a statistics lecturer at the London School of Economics. 'If the coin were unbiased, the chance of getting a result as extreme as that would be less than 7%.'

"But [MacKay asks] do these data give evidence that the coin is biased rather than fair?"

To answer that question, we'll proceed in two steps. First we'll use the binomial distribution to see where that 7% came from; then we'll use Bayes's Theorem to estimate the probability that this coin comes up heads.

The Binomial Distribution

Suppose I tell you that a coin is "fair", that is, the probability of heads is 50%. If you spin it twice, there are four outcomes: HH, HT, TH, and TT. All four outcomes have the same probability, 25%.

If we add up the total number of heads, there are three possible results: 0, 1, or 2. The probabilities of 0 and 2 are 25%, and the probability of 1 is 50%.

More generally, suppose the probability of heads is p and we spin the coin n times. The probability that we get a total of k heads is given by the binomial distribution:

$$\binom{n}{k} p^k (1 - p)^{n-k}$$

for any value of k from 0 to n , including both. The term $\binom{n}{k}$ is the binomial coefficient, usually pronounced "n choose k".

We could evaluate this expression ourselves, but we can also use the SciPy function `binom.pmf`. For example, if we flip a coin `n=2` times and the probability of heads is `p=0.5`, here's the probability of getting `k=1` heads:

```
from scipy.stats import binom

n = 2
p = 0.5
k = 1

binom.pmf(k, n, p)
```

```
np.float64(0.5000000000000002)
```

Instead of providing a single value for `k`, we can also call `binom.pmf` with an array of values.

```
import numpy as np
ks = np.arange(n+1)

ps = binom.pmf(ks, n, p)
ps
```

```
array([0.25, 0.5 , 0.25])
```

The result is a NumPy array with the probability of 0, 1, or 2 heads. If we put these probabilities in a `Pmf`, the result is the distribution of `k` for the given values of `n` and `p`.

Here's what it looks like:

```
from empiricaldist import Pmf

pmf_k = Pmf(ps, ks)
pmf_k
```

	probs
0	0.25
1	0.50
2	0.25

The following function computes the binomial distribution for given values of `n` and `p` and returns a `Pmf` that represents the result.

```
def make_binomial(n, p):
    """Make a binomial Pmf."""
    ks = np.arange(n+1)
    ps = binom.pmf(ks, n, p)
    return Pmf(ps, ks)
```

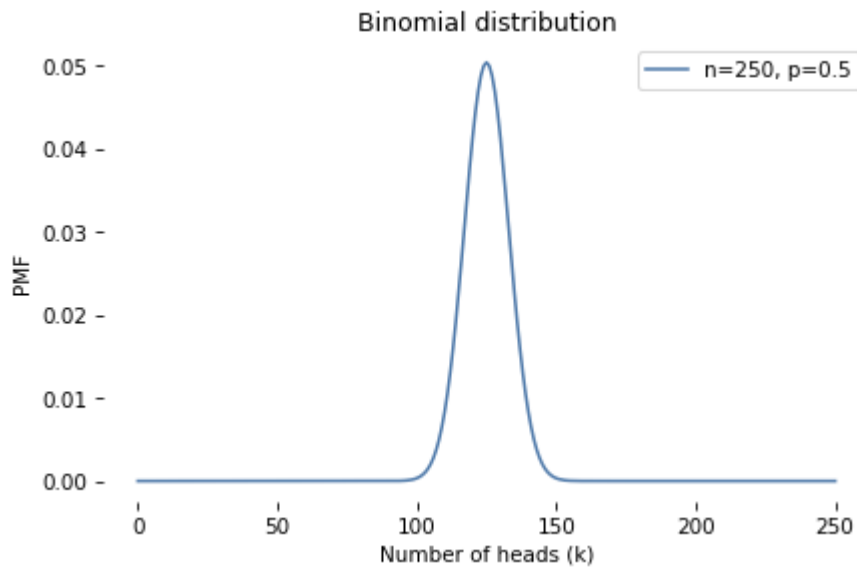
Here's what it looks like with `n=250` and `p=0.5`:

```
pmf_k = make_binomial(n=250, p=0.5)
```

```
from utils import decorate

pmf_k.plot(label='n=250, p=0.5')

decorate(xlabel='Number of heads (k)',
        ylabel='PMF',
        title='Binomial distribution')
```



The most likely quantity in this distribution is 125:

```
pmf_k.max_prob()
```

```
np.int64(125)
```

But even though it is the most likely quantity, the probability that we get exactly 125 heads is only about 5%.

```
pmf_k[125]
```

```
np.float64(0.050412213147309655)
```

In MacKay's example, we got 140 heads, which is even less likely than 125:

```
pmf_k[140]
```

```
np.float64(0.008357181724918205)
```

In the article MacKay quotes, the statistician says, "If the coin were unbiased the chance of getting a result as extreme as that would be less than 7%."

We can use the binomial distribution to check his math. The following function takes a PMF and computes the total probability of quantities greater than or equal to `threshold`.

```
def prob_ge(pmf, threshold):
    """Probability of quantities greater than threshold."""
    ge = (pmf.qs >= threshold)
    total = pmf[ge].sum()
    return total
```

Here's the probability of getting 140 heads or more:

```
prob_ge(pmf_k, 140)
```

```
np.float64(0.03321057562002168)
```

`Pmf` provides a method that does the same computation.

```
pmf_k.prob_ge(140)
```

```
np.float64(0.03321057562002168)
```

The result is about 3.3%, which is less than the quoted 7%. The reason for the difference is that the statistician includes all outcomes "as extreme as" 140, which includes outcomes less than or equal to 110.

To see where that comes from, recall that the expected number of heads is 125. If we get 140, we've exceeded that expectation by 15. And if we get 110, we have come up short by 15.

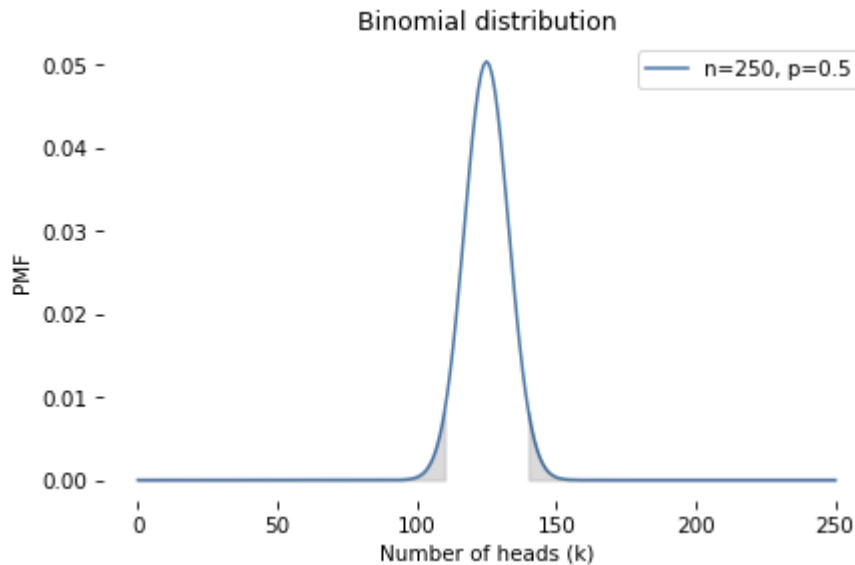
7% is the sum of both of these "tails", as shown in the following figure.

```
import matplotlib.pyplot as plt

def fill_below(pmf):
    qs = pmf.index
    ps = pmf.values
    plt.fill_between(qs, ps, 0, color='C5', alpha=0.4)

qs = pmf_k.index
fill_below(pmf_k[qs>=140])
fill_below(pmf_k[qs<=110])
pmf_k.plot(label='n=250, p=0.5')

decorate(xlabel='Number of heads (k)',
         ylabel='PMF',
         title='Binomial distribution')
```



Here's how we compute the total probability of the left tail.

```
pmf_k.prob_le(110)
```

```
np.float64(0.03321057562002167)
```

The probability of outcomes less than or equal to 110 is also 3.3%, so the total probability of outcomes "as extreme" as 140 is 6.6%.

The point of this calculation is that these extreme outcomes are unlikely if the coin is fair.

That's interesting, but it doesn't answer MacKay's question. Let's see if we can.

Bayesian Estimation

Any given coin has some probability of landing heads up when spun on edge; I'll call this probability x . It seems reasonable to believe that x depends on physical characteristics of the coin, like the distribution of weight. If a coin is perfectly balanced, we expect x to be close to 50%, but for a lopsided coin, x might be substantially different. We can use Bayes's theorem and the observed data to estimate x .

For simplicity, I'll start with a uniform prior, which assumes that all values of x are equally likely. That might not be a reasonable assumption, so we'll come back and consider other priors later.

We can make a uniform prior like this:

```
hypos = np.linspace(0, 1, 101)
prior = Pmf(1, hypos)
```

`hypos` is an array of equally spaced values between 0 and 1.

We can use the hypotheses to compute the likelihoods, like this:

```
likelihood_heads = hypos
likelihood_tails = 1 - hypos
```

I'll put the likelihoods for heads and tails in a dictionary to make it easier to do the update.

```
likelihood = {
    'H': likelihood_heads,
    'T': likelihood_tails
}
```

To represent the data, I'll construct a string with `H` repeated 140 times and `T` repeated 110 times.

```
dataset = 'H' * 140 + 'T' * 110
```

The following function does the update.

```
def update_euro(pmf, dataset):
    """Update pmf with a given sequence of H and T."""
    for data in dataset:
        pmf *= likelihood[data]

    pmf.normalize()
```

The first argument is a `Pmf` that represents the prior. The second argument is a sequence of strings. Each time through the loop, we multiply `pmf` by the likelihood of one outcome, `H` for heads or `T` for tails.

Notice that `normalize` is outside the loop, so the posterior distribution only gets normalized once, at the end. That's more efficient than normalizing it after each spin (although we'll see later that it can also cause problems with floating-point arithmetic).

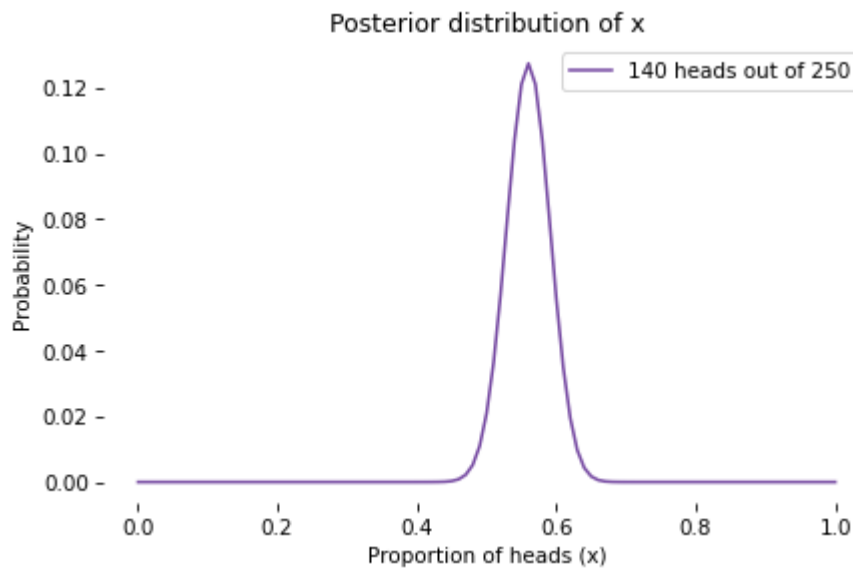
Here's how we use `update_euro`.

```
posterior = prior.copy()
update_euro(posterior, dataset)
```

And here's what the posterior looks like.

```
def decorate_euro(title):
    decorate(xlabel='Proportion of heads (x)',
             ylabel='Probability',
             title=title)
```

```
posterior.plot(label='140 heads out of 250', color='C4')
decorate_euro(title='Posterior distribution of x')
```



This figure shows the posterior distribution of x , which is the proportion of heads for the coin we observed.

The posterior distribution represents our beliefs about x after seeing the data. It indicates that values less than 0.4 and greater than 0.7 are unlikely; values between 0.5 and 0.6 are the most likely.

In fact, the most likely value for x is 0.56 which is the proportion of heads in the dataset, 140/250.

```
posterior.max_prob()
```

```
np.float64(0.56)
```

Triangle Prior

So far we've been using a uniform prior:

```
uniform = Pmf(1, hypos, name='uniform')
uniform.normalize()
```



```
np.int64(101)
```

But that might not be a reasonable choice based on what we know about coins. I can believe that if a coin is lopsided, x might deviate substantially from 0.5, but it seems unlikely that the Belgian Euro coin is so imbalanced that x is 0.1 or 0.9.

It might be more reasonable to choose a prior that gives higher probability to values of x near 0.5 and lower probability to extreme values.

As an example, let's try a triangle-shaped prior. Here's the code that constructs it:

```
ramp_up = np.arange(50)
ramp_down = np.arange(50, -1, -1)

a = np.append(ramp_up, ramp_down)

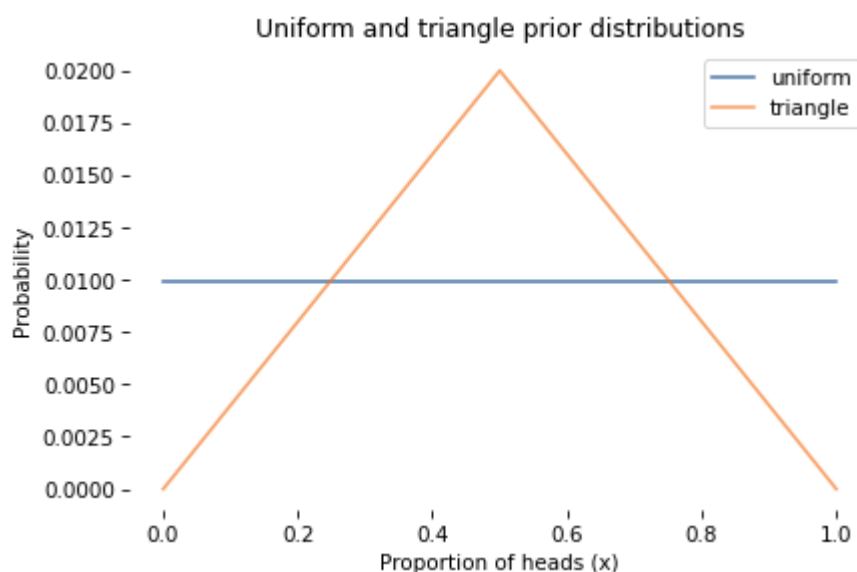
triangle = Pmf(a, hypos, name='triangle')
triangle.normalize()
```

```
np.int64(2500)
```

`arange` returns a NumPy array, so we can use `np.append` to append `ramp_down` to the end of `ramp_up`. Then we use `a` and `hypos` to make a `Pmf`.

The following figure shows the result, along with the uniform prior.

```
uniform.plot()
triangle.plot()
decorate_euro(title='Uniform and triangle prior distributions')
```

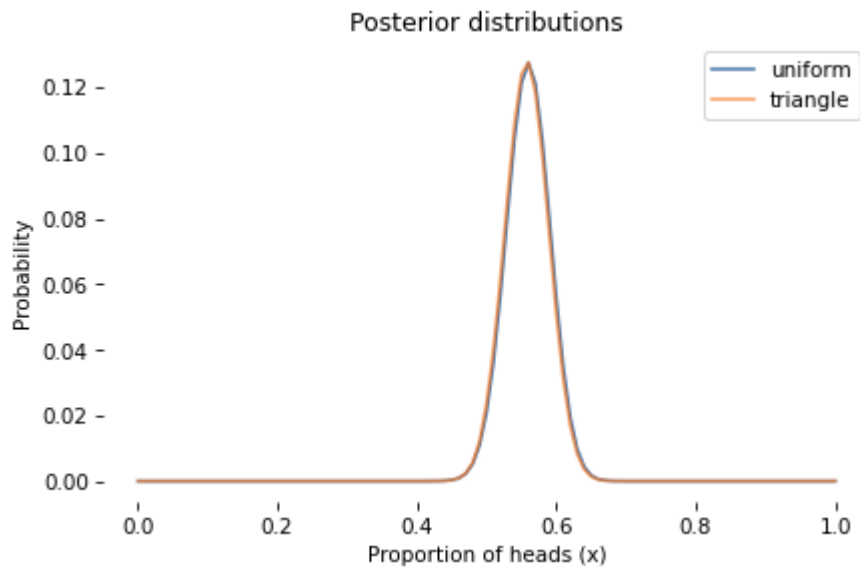


Now we can update both priors with the same data:

```
update_euro(uniform, dataset)
update_euro(triangle, dataset)
```

Here are the posteriors.

```
uniform.plot()
triangle.plot()
decorate_euro(title='Posterior distributions')
```



The differences between the posterior distributions are barely visible, and so small they would hardly matter in practice.

And that's good news. To see why, imagine two people who disagree angrily about which prior is better, uniform or triangle. Each of them has reasons for their preference, but neither of them can persuade the other to change their mind.

But suppose they agree to use the data to update their beliefs. When they compare their posterior distributions, they find that there is almost nothing left to argue about.

This is an example of **swamping the priors**: with enough data, people who start with different priors will tend to converge on the same posterior distribution.

The Binomial Likelihood Function

So far we've been computing the updates one spin at a time, so for the Euro problem we have to do 250 updates.

A more efficient alternative is to compute the likelihood of the entire dataset at once. For each hypothetical value of x , we have to compute the probability of getting 140 heads out of 250 spins.

Well, we know how to do that; this is the question the binomial distribution answers. If the probability of heads is p , the probability of k heads in n spins is:

$$\binom{n}{k} p^k (1 - p)^{n-k}$$

And we can use SciPy to compute it. The following function takes a `Pmf` that represents a prior distribution and a tuple of integers that represent the data:

```
from scipy.stats import binom

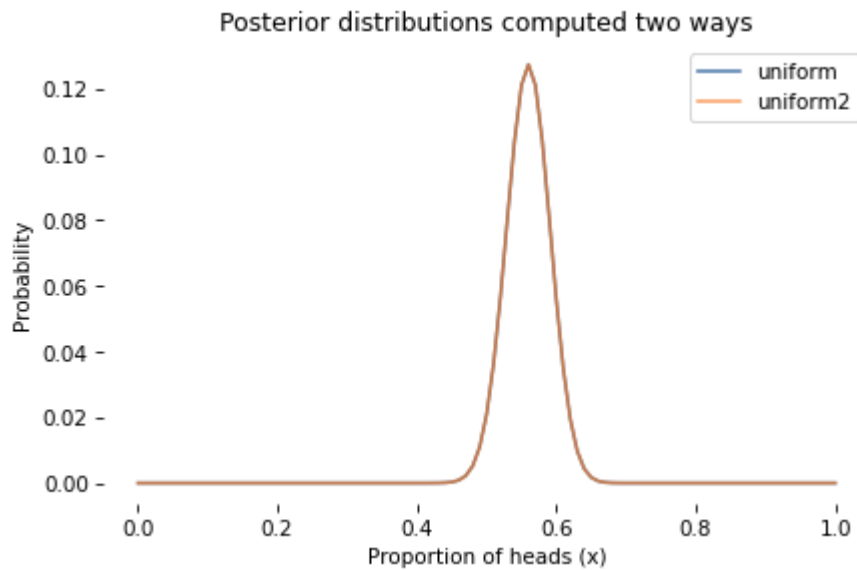
def update_binomial(pmf, data):
    """Update pmf using the binomial distribution."""
    k, n = data
    xs = pmf.qs
    likelihood = binom.pmf(k, n, xs)
    pmf *= likelihood
    pmf.normalize()
```

The data are represented with a tuple of values for `k` and `n`, rather than a long string of outcomes. Here's the update.

```
uniform2 = Pmf(1, hypos, name='uniform2')
data = 140, 250
update_binomial(uniform2, data)
```

And here's what the posterior looks like.

```
uniform.plot()
uniform2.plot()
decorate_euro(title='Posterior distributions computed two ways')
```



We can use `allclose` to confirm that the result is the same as in the previous section except for a small floating-point round-off.

```
np.allclose(uniform, uniform2)
```

```
True
```

But this way of doing the computation is much more efficient.

Bayesian Statistics

You might have noticed similarities between the Euro problem and the 101 Bowls Problem in `<<_101Bowls>>`. The prior distributions are the same, the likelihoods are the same, and with the same data the results would be the same. But there are two differences.

The first is the choice of the prior. With 101 bowls, the uniform prior is implied by the statement of the problem, which says that we choose one of the bowls at random with equal probability.

In the Euro problem, the choice of the prior is subjective; that is, reasonable people could disagree, maybe because they have different information about coins or because they interpret the same information differently.

Because the priors are subjective, the posteriors are subjective, too. And some people find that problematic.

The other difference is the nature of what we are estimating. In the 101 Bowls problem, we choose the bowl randomly, so it is uncontroversial to compute the probability of choosing each bowl. In the Euro problem, the proportion of heads is a physical property of a given coin. Under some interpretations of probability, that's a problem because physical properties are not considered random.

As an example, consider the age of the universe. Currently, our best estimate is 13.80 billion years, but it might be off by 0.02 billion years in either direction (see [here](#)).

Now suppose we would like to know the probability that the age of the universe is actually greater than 13.81 billion years. Under some interpretations of probability, we would not be able to answer that question. We would be required to say something like, "The age of the universe is not a random quantity, so it has no probability of exceeding a particular value."

Under the Bayesian interpretation of probability, it is meaningful and useful to treat physical quantities as if they were random and compute probabilities about them.

In the Euro problem, the prior distribution represents what we believe about coins in general and the posterior distribution represents what we believe about a particular coin after seeing the data. So we can use the posterior distribution to compute probabilities about the coin and its proportion of heads.

The subjectivity of the prior and the interpretation of the posterior are key differences between using Bayes's Theorem and doing Bayesian statistics.

Bayes's Theorem is a mathematical law of probability; no reasonable person objects to it. But Bayesian statistics is surprisingly controversial. Historically, many people have been bothered by its subjectivity and its use of probability for things that are not random.

If you are interested in this history, I recommend Sharon Bertsch McGrayne's book, *The Theory That Would Not Die*.

Summary

In this chapter I posed David MacKay's Euro problem and we started to solve it. Given the data, we computed the posterior distribution for `x`, the probability a Euro coin comes up heads.

We tried two different priors, updated them with the same data, and found that the posteriors were nearly the same. This is good news, because it suggests that if two people start with different beliefs and see the same data, their beliefs tend to converge.

This chapter introduces the binomial distribution, which we used to compute the posterior distribution more efficiently. And I discussed the differences between applying Bayes's Theorem, as in the 101 Bowls problem, and doing Bayesian statistics, as in the Euro problem.

However, we still haven't answered MacKay's question: "Do these data give evidence that the coin is biased rather than fair?" I'm going to leave this question hanging a little longer; we'll come back to it in `<<_Testing>>`.

In the next chapter, we'll solve problems related to counting, including trains, tanks, and rabbits.

But first you might want to work on these exercises.

Exercises

Exercise: In Major League Baseball, most players have a batting average between .200 and .330, which means that their probability of getting a hit is between 0.2 and 0.33.

Suppose a player appearing in their first game gets 3 hits out of 3 attempts. What is the posterior distribution for their probability of getting a hit?

For this exercise, I'll construct the prior distribution by starting with a uniform distribution and updating it with imaginary data until it has a shape that reflects my background knowledge of batting averages.

Here's the uniform prior:

```
hypos = np.linspace(0.1, 0.4, 101)
prior = Pmf(1, hypos)
```

And here is a dictionary of likelihoods, with `Y` for getting a hit and `N` for not getting a hit.

```
likelihood = {
    'Y': hypos,
    'N': 1-hypos
}
```

Here's a dataset that yields a reasonable prior distribution.

```
dataset = 'Y' * 25 + 'N' * 75
```

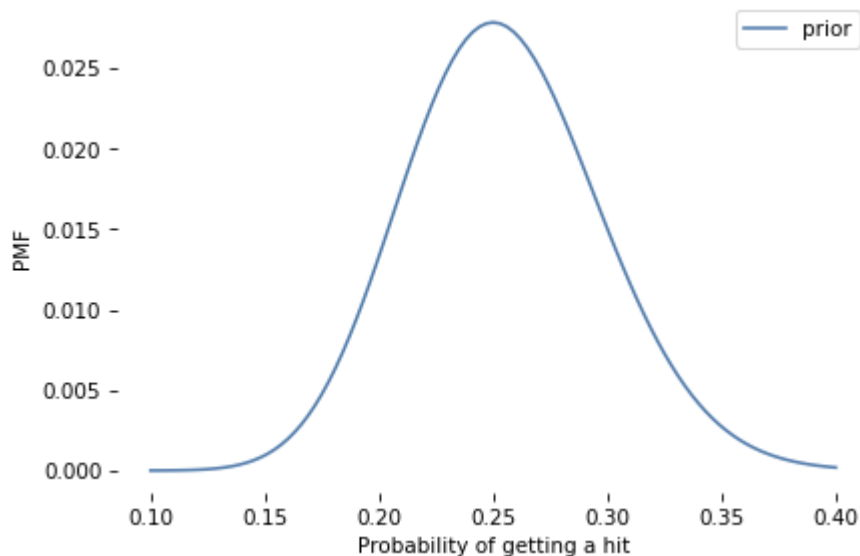
And here's the update with the imaginary data.

```
for data in dataset:  
    prior *= likelihood[data]  
  
prior.normalize()
```

```
np.float64(1.3596964206970507e-23)
```

Finally, here's what the prior looks like.

```
prior.plot(label='prior')  
decorate(xlabel='Probability of getting a hit',  
         ylabel='PMF')
```



This distribution indicates that most players have a batting average near 250, with only a few players below 175 or above 350. I'm not sure how accurately this prior reflects the distribution of batting averages in Major League Baseball, but it is good enough for this exercise.

Now update this distribution with the data and plot the posterior. What is the most likely quantity in the posterior distribution?

```
# Solution

posterior = prior.copy()

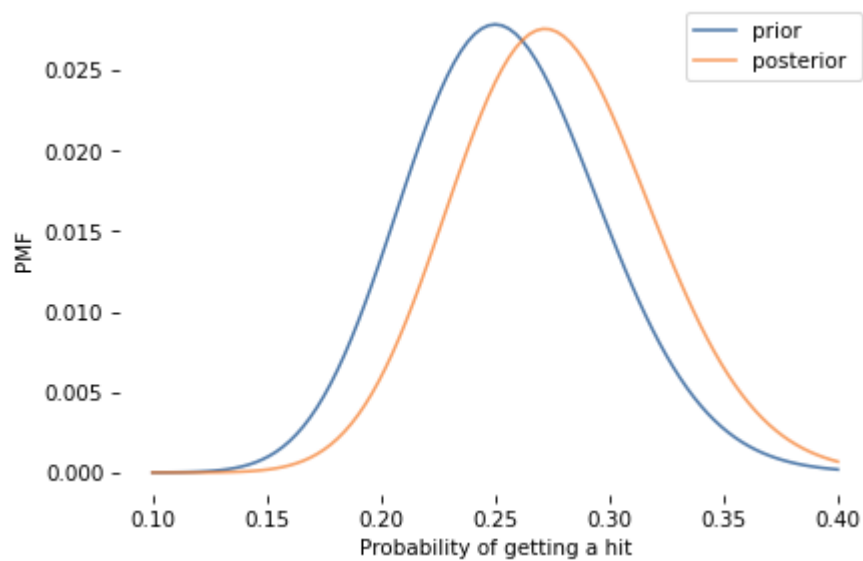
for data in 'YYY':
    posterior *= likelihood[data]

posterior.normalize()
```

```
np.float64(0.017944179687707326)
```

```
# Solution

prior.plot(label='prior')
posterior.plot(label='posterior ')
decorate(xlabel='Probability of getting a hit',
         ylabel='PMF')
```



```
# Solution

prior.max_prob()
```

```
np.float64(0.25)
```

```
# Solution

posterior.max_prob()
```

```
np.float64(0.271)
```


Exercise: Whenever you survey people about sensitive issues, you have to deal with social desirability bias, which is the tendency of people to adjust their answers to show themselves in the most positive light. One way to improve the accuracy of the results is randomized response.

As an example, suppose you want to know how many people cheat on their taxes. If you ask them directly, it is likely that some of the cheaters will lie. You can get a more accurate estimate if you ask them indirectly, like this: Ask each person to flip a coin and, without revealing the outcome,

- If they get heads, they report YES.
- If they get tails, they honestly answer the question "Do you cheat on your taxes?"

If someone says YES, we don't know whether they actually cheat on their taxes; they might have flipped heads. Knowing this, people might be more willing to answer honestly.

Suppose you survey 100 people this way and get 80 YESes and 20 NOs. Based on this data, what is the posterior distribution for the fraction of people who cheat on their taxes? What is the most likely quantity in the posterior distribution?

Solution

*# I'll use a uniform distribution again, although there might
be background information we could use to choose a more
specific prior.*

```
hypos = np.linspace(0, 1, 101)
prior = Pmf(1, hypos)
```

Solution

*# If the actual fraction of cheaters is `x`, the number of
YESes is $(0.5 + x/2)$, and the number of NOs is $(1-x)/2$*

```
likelihood = {
    'Y': 0.5 + hypos/2,
    'N': (1-hypos)/2
}
```

Solution

```
dataset = 'Y' * 80 + 'N' * 20

posterior = prior.copy()

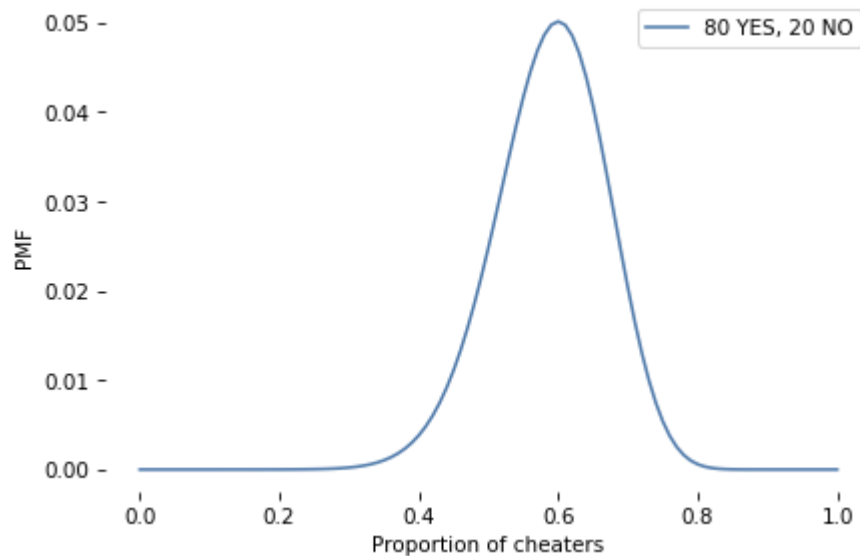
for data in dataset:
    posterior *= likelihood[data]

posterior.normalize()
```

```
np.float64(3.6945139133967024e-21)
```

```
# Solution
```

```
posterior.plot(label='80 YES, 20 NO')  
decorate(xlabel='Proportion of cheaters',  
         ylabel='PMF')
```



```
# Solution
```

```
posterior.idxmax()
```

```
np.float64(0.6)
```

Exercise: Suppose you want to test whether a coin is fair, but you don't want to spin it hundreds of times. So you make a machine that spins the coin automatically and uses computer vision to determine the outcome.

However, you discover that the machine is not always accurate. Specifically, suppose the probability is $y=0.2$ that an actual heads is reported as tails, or actual tails reported as heads.

If we spin a coin 250 times and the machine reports 140 heads, what is the posterior distribution of x ? What happens as you vary the value of y ?

Solution

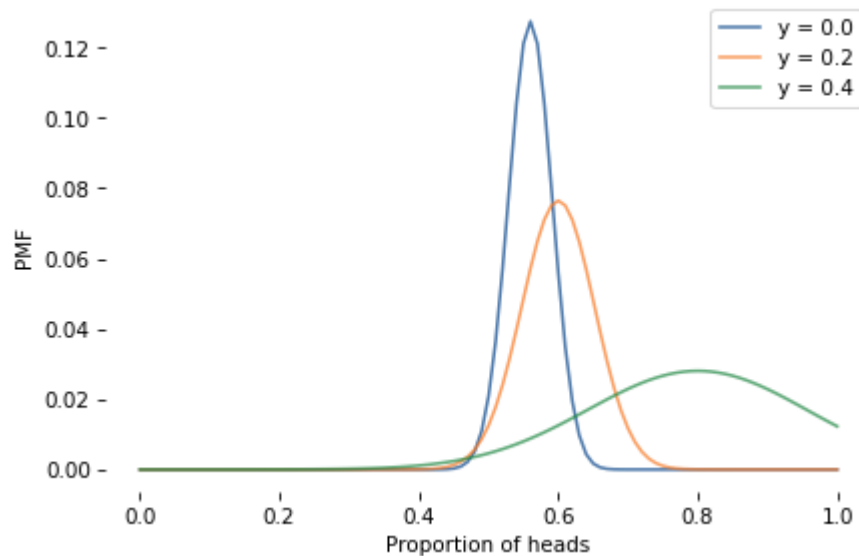
```
def update_unreliable(pmf, dataset, y):  
    likelihood = {  
        'H': (1-y) * hypos + y * (1-hypos),  
        'T': y * hypos + (1-y) * (1-hypos)  
    }  
    for data in dataset:  
        pmf *= likelihood[data]  
  
    pmf.normalize()
```

Solution

```
hypos = np.linspace(0, 1, 101)  
prior = Pmf(1, hypos)  
dataset = 'H' * 140 + 'T' * 110  
  
posterior00 = prior.copy()  
update_unreliable(posterior00, dataset, 0.0)  
  
posterior02 = prior.copy()  
update_unreliable(posterior02, dataset, 0.2)  
  
posterior04 = prior.copy()  
update_unreliable(posterior04, dataset, 0.4)
```

Solution

```
posterior00.plot(label='y = 0.0')  
posterior02.plot(label='y = 0.2')  
posterior04.plot(label='y = 0.4')  
decorate(xlabel='Proportion of heads',  
         ylabel='PMF')
```



Solution

```
posterior00.idxmax(), posterior02.idxmax(), posterior04.idxmax()
```

```
(np.float64(0.56), np.float64(0.6), np.float64(0.8))
```

Exercise: In preparation for an alien invasion, the Earth Defense League (EDL) has been working on new missiles to shoot down space invaders. Of course, some missile designs are better than others; let's assume that each design has some probability of hitting an alien ship, x .

Based on previous tests, the distribution of x in the population of designs is approximately uniform between 0.1 and 0.4.

Now suppose the new ultra-secret Alien Blaster 9000 is being tested. In a press conference, an EDL general reports that the new design has been tested twice, taking two shots during each test. The results of the test are confidential, so the general won't say how many targets were hit, but they report: "The same number of targets were hit in the two tests, so we have reason to think this new design is consistent."

Is this data good or bad? That is, does it increase or decrease your estimate of x for the Alien Blaster 9000?

Hint: If the probability of hitting each target is x , the probability of hitting one target in both tests is $[2x(1 - x)]^2$.

Solution

```
hypos = np.linspace(0.1, 0.4, 101)
prior = Pmf(1, hypos)
```

```
# Solution

# Here's a specific version for n=2 shots per test

x = hypos
likes = [(1-x)**4, (2*x*(1-x))**2, x**4]
likelihood = np.sum(likes, axis=0)
```

```
# Solution

# Here's a more general version for any n shots per test

from scipy.stats import binom

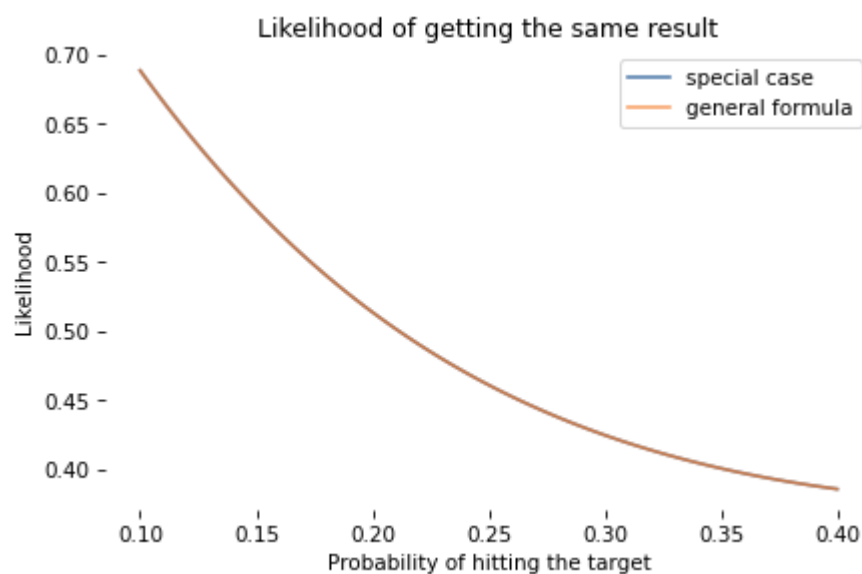
n = 2
likes2 = [binom.pmf(k, n, x)**2 for k in range(n+1)]
likelihood2 = np.sum(likes2, axis=0)
```

```
# Solution

# Here are the likelihoods, computed both ways

import matplotlib.pyplot as plt

plt.plot(x, likelihood, label='special case')
plt.plot(x, likelihood2, label='general formula')
decorate(xlabel='Probability of hitting the target',
         ylabel='Likelihood',
         title='Likelihood of getting the same result')
```



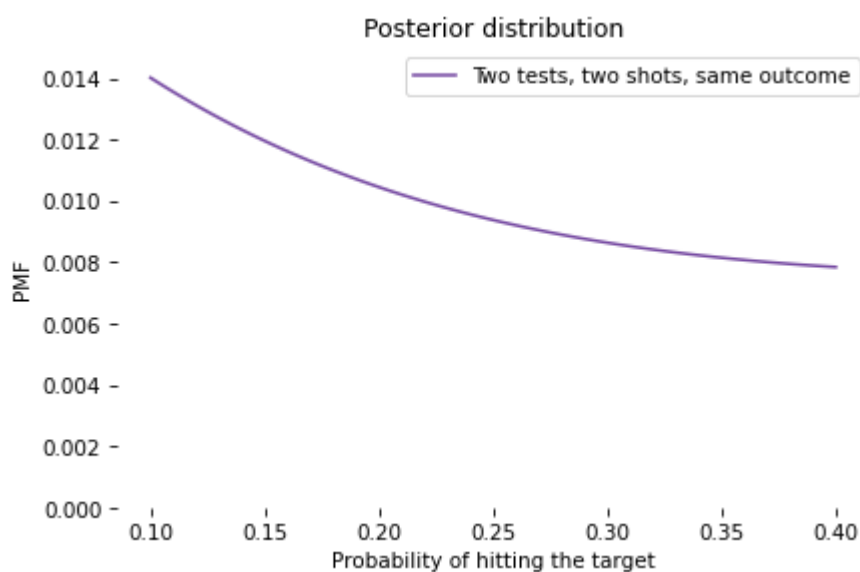
```
# Solution
```

```
posterior = prior * likelihood
posterior.normalize()
```

```
np.float64(49.129627998379995)
```

```
# Solution
```

```
posterior.plot(label='Two tests, two shots, same outcome',
               color='C4')
decorate(xlabel='Probability of hitting the target',
        ylabel='PMF',
        title='Posterior distribution',
        ylim=[0, 0.015])
```



```
# Solution
```

```
# Getting the same result in both tests is more likely for
# extreme values of `x` and least likely when `x=0.5`.
```

```
# In this example, the prior indicates that `x` is less than 0.5,
# and the update gives more weight to extreme values.
```

```
# So the dataset makes lower values of `x` more likely.
```

Think Bayes, Second Edition

Copyright 2020 Allen B. Downey

License: Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0)